

Makefile - Essential Summary

What is a Makefile?

A Makefile is a file used to automate compilation and build tasks.

Instead of writing:

 Bash 

```
cc -Wall -Wextra -Werror -c main.c
cc -Wall -Wextra -Werror -c utils.c
ar rcs libft.a main.o utils.o
```

You write:

 Bash 

```
make
```

Rule

A rule tells `make` how to build something.

Syntax:

 Makefile 

```
target: dependencies
    command
```

Example:

 Makefile 

```
libft.a: main.o utils.o
    ar rcs libft.a main.o utils.o
```

Meaning:

To build libft.a, main.o and utils.o are required. 

Target

The thing we want to build.

Examples:

</> Makefile



```
libft.a  
all  
clean  
fclean  
re  
main.o
```

Dependency

A file needed before building a target.

Example:

</> Makefile



```
main.o: main.c
```

Meaning:

```
main.o depends on main.c
```



Source File

.c file



Example:

```
main.c  
utils.c
```



Object File

.o file



Created from:

```
.c -> .o
```



Example:

```
main.o  
utils.o
```



Compile

Convert:

```
.c -> .o
```



Example:

```
</> Bash
```



```
cc -c main.c
```

Static Library

A collection of object files.

Example:

```
libft.a
```



ar Command

Creates a static library.

Example:

```
</> Bash
```



```
ar rcs libft.a main.o utils.o
```

Options

```
r = replace/add files
c = create library
s = create symbol table
```



Variables

Store values to avoid repetition.

Example:

Makefile



```
CC = cc

CFLAGS = -Wall -Wextra -Werror

NAME = libft.a

SRCS = main.c utils.c

OBJS = main.o utils.o
```

Use:

Makefile



```
$(CC)
$(CFLAGS)
$(NAME)
$(SRCS)
$(OBJS)
```

Automatic Object Generation

Instead of:

Makefile



```
OBJS = main.o utils.o
```

Use:

Makefile



```
OBJS = $(SRCS:.c=.o)
```

Meaning:

```
Replace .c with .o
```



Example:

```
</> Makefile
```



```
SRCS = main.c utils.c
```

Becomes:

```
</> Makefile
```



```
OBJS = main.o utils.o
```

Pattern Rule

Generic rule for building all `.o` files.

```
</> Makefile
```



```
%.o: %.c  
    cc $(CFLAGS) -c $< -o $@
```

Meaning:

```
Any .o file can be built from a .c file  
with the same name.
```



Example:

```
main.o -> main.c  
utils.o -> utils.c
```



Automatic Variables

```
$@
```

Current target.

Example:

</> Makefile



```
main.o: main.c
```

```
$@ = main.o
```



\$<

First dependency.

Example:

</> Makefile



```
main.o: main.c
```

```
$< = main.c
```



Example:

</> Makefile



```
cc -c $< -o $@
```

Becomes:

</> Bash



```
cc -c main.c -o main.o
```

all

Default rule executed by:

</> Bash



```
make
```

Example:

</> Makefile



```
all: libft.a
```

Meaning:

```
make == make all
```



clean

Removes temporary files.

Example:

</> Makefile



```
clean:
    rm -f *.o
```

Removes:

```
.o files
```



fclean

Removes everything generated.

Example:

</> Makefile



```
fclean: clean
    rm -f libft.a
```

Removes:

```
.o files
libft.a
```



re

Rebuild everything.

Example:

Makefile

```
re: fclean all
```



Equivalent to:

Bash

```
make fclean
make all
```



.PHONY

Makefile

```
.PHONY: all clean fclean re
```



Meaning:

```
These targets are commands, not real files.
Always run them.
```



Prevents conflicts with files named:

```
all
clean
fclean
re
```



Wildcards

Special characters that match multiple files.

*

Bash



`*.c`

Meaning:

Any file ending with `.c`



Example:

```
main.c
utils.c
test.c
```



?

`</>` Bash



`file?.c`

Meaning:

Exactly one character



Matches:

```
file1.c
fileA.c
```



Not:

`file10.c`



`[]`

`</>` Bash



`file[123].c`

Matches:

```
file1.c
file2.c
```



file3.c

Dependency Tracking

Make checks timestamps.

If:

```
main.c is newer than main.o
```



Recompile:

```
main.o
```



If:

```
main.o is up to date
```



Do nothing.

Important Goal of Make

```
Compile only what changed.  
Avoid unnecessary commands.  
Reuse existing build files when possible.
```



Minimal Makefile Template

<> Makefile



```
NAME = libft.a  
  
CC = cc  
CFLAGS = -Wall -Wextra -Werror  
  
SRCS = main.c utils.c  
OBJS = $(SRCS:.c=.o)  
  
all: $(NAME)  
  
$(NAME): $(OBJS)  
    ar rcs $(NAME) $(OBJS)
```

```
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJS)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

Terms

- **Target:** File or command to build.
- **Dependency:** Required file before building a target.
- **Source File:** `.c` file.
- **Object File:** `.o` file.
- **Compile:** Convert `.c` to `.o`.
- **Static Library:** `.a` file containing object files.
- **Pattern Rule:** Generic rule for multiple files.
- **Wildcard:** Pattern matching multiple filenames.
- **Timestamp:** File modification time used by make.
- **PHONY Target:** Command target that is not a real file.